

NEARFIX - HYPERLOCAL ON-DEMAND HOME SERVICE MARKETPLACE

PROJECT THESIS

SUBMITTED

TO

**AWH ENGINEERING COLLEGE
KUTTIKKATTOOR, CALICUT**

**IN PARTIAL FULFILMENT
OF THE REQUIREMENTS FOR THE AWARD OF THE DEGREE
OF**

Master Of Computer Applications

BY

ABHAY C BABU



**DEPARTMENT OF COMPUTER APPLICATIONS
AWH ENGINEERING COLLEGE KUTTIKKATTOOR
CALICUT
NOVEMBER 2024**

DEPARTMENT OF COMPUTER APPLICATIONS



**AWH ENGINEERING COLLEGE
CALICUT**

CERTIFICATE

This is to certify that this thesis entitled “NEARFIX - HYPERLOCAL ON-DEMAND HOME SERVICE MARKETPLACE” submitted herewith is an authentic record of the thesis work done by ABHAY C BABU (AWH23MCA-2001) under our guidance in partial fulfillment of the requirements for the award of Master of Computer Applications from APJ Abdul Kalam Technological University during the academic year 2024–2025.

Mrs. Sruti Sudevan
Associate professor
Dept. Of computer applications

Head of the Department

Mrs. Aiswarya N
Assistant professor
Dept. Of computer applications

Project Guide

External Examiner

Internal Examiner

ACKNOWLEDGEMENT

I express my sincere gratitude to our beloved principal **Dr. Sabeena M V** for providing me an opportunity with the required facilities for doing this project. I express my hearty thanks to **Mrs. Sruti Sudevan**, Head of the Department of Computer Applications, and **Mrs. Aiswarya N**, Assistant Professor, for their invaluable guidance, encouragement, and persistent direction throughout the development of this project.

I am thankful to all other staff members of the MCA department for their encouragement, timely guidance, valuable suggestions, and inspiring ideas given throughout this project.

I am grateful to my friends and family for the way they have cooperated, motivated me to achieve success, and continuously stirred my ambition to produce the finest outcome.

Above all, I am grateful to the Almighty, who has showered His abundant blessings on me throughout my life and throughout the journey of completing this project.

ABHAY C BABU

ABSTRACT

This project report presents an innovative, reliable, and scalable web platform titled **NearFix**, designed to bridge the structural divide between skilled micro-entrepreneurs (electricians, plumbers, carpenters, HVAC technicians) and local residents seeking dependable on-demand home services. In modern urban and semi-urban landscapes, locating credible local technicians is burdened by opaque pricing, lack of verification, and prolonged response latency during domestic emergencies.

NearFix introduces an integrated dual-booking mechanism that provides both standard calendar-scheduled reservations and priority emergency dispatch with real-time surge pricing transparency. Utilizing open-source Leaflet and OpenStreetMap rendering, users can visually explore nearby verified service professionals on an interactive map. Safety and reliability are reinforced through an administrative Know-Your-Customer (KYC) audit pipeline requiring technicians to submit government identity documents, skill certifications, and work portfolios prior to public activation.

Built upon a robust, decoupled architecture consisting of a React 19 single-page frontend, a Node.js Express REST API backend, and a flexible MongoDB document repository, the platform provides seamless session handling, automated status updates, dispute management, and transparent customer review scoring. The solution empowers local skilled workers with independent digital storefronts while granting consumers an accessible, secure, and modern utility ecosystem.

CONTENTS

	Page No
1. INTRODUCTION	1
2. SYSTEM ANALYSIS	3
2.1 Existing System	4
2.2 Proposed System	4
2.3 Module Description	5
2.4 Sprint	6
2.5 User Stories	8
3. FEASIBILITY STUDY	9
3.1 Economic Feasibility	10
3.2 Technical Feasibility	10
3.3 Operational Feasibility	10
3.4 Behavioral Feasibility	11
3.5 Software Feasibility	11
4. SOFTWARE ENGINEERING PARADIGM	12
4.1 Agile model	13
4.2 Scrum	13
5. SYSTEM REQUIREMENT SPECIFICATIONS	14
5.1 Software Requirements	15
5.2 Hardware Requirements	15
6. SYSTEM DESIGN	16
6.1 Database Design	17
6.2 Tables	18
6.3 UML Design	22
6.4 Use Case Diagram	23
6.5 Scenario	24
6.6 Sequence Diagram	25

7. SYSTEM DEVELOPMENT	28
7.1 Coding	29
8. SYSTEM TESTING AND IMPLEMENTATION	31
8.1 Types of Testing	32
8.2 Implementation	32
9. SYSTEM MAINTENANCE	33
10. FUTURE ENHANCEMENT	36
11. CONCLUSION	37
12. APPENDIX	39
13. BIBLIOGRAPHY	46

INTRODUCTION

1. INTRODUCTION

In today's fast-paced society, urban and suburban households frequently encounter unpredictable domestic maintenance emergencies ranging from electrical short circuits and plumbing ruptures to home appliance malfunctions. Concurrently, skilled independent tradespeople—including electricians, plumbers, painters, carpenters, and appliance mechanics—frequently struggle to secure stable livelihoods due to reliance on unorganized word-of-mouth networks and the lack of accessible digital marketing platforms.

Recognizing this critical market gap, this project focuses on the conceptualization, architectural design, and implementation of **NearFix**, a full-stack hyperlocal on-demand home service marketplace. NearFix establishes a direct, frictionless, and secure digital channel uniting neighborhood service seekers with thoroughly verified local technicians. By eliminating traditional intermediaries, the platform empowers micro-entrepreneurs while providing homeowners with unprecedented convenience, transparency, and dependability.

The NearFix platform incorporates an intuitive user experience featuring geospatial worker discovery powered by open-source interactive mapping. Customers can visually inspect technician locations, view verified credentials, examine hourly and project-based pricing, and read authentic client reviews before confirming a booking. A distinguished feature of NearFix is its dual-mode booking architecture: users can schedule standard appointments in advance or trigger high-priority emergency dispatches with automated surge rate calculations for time-critical household breakdowns.

Furthermore, trust and security serve as the cornerstone of the platform. NearFix integrates a mandatory administrative Know-Your-Customer (KYC) vetting workflow where technicians must provide legitimate government identity documents, proof of address, and professional certifications prior to marketplace onboarding. Combined with transparent billing, lifecycle status tracking, and a robust administrative dispute resolution system, NearFix redefines local service delivery into a modern, professional, and reliable digital ecosystem.

SYSTEM ANALYSIS

2. SYSTEM ANALYSIS

2.1 Existing system

The prevailing mechanism for acquiring domestic home services relies predominantly on informal, localized channels such as physical business cards, telephone directories, neighborhood notice boards, or informal recommendations from acquaintances. This manual paradigm suffers from profound structural inefficiencies. Customers have zero visibility into a technician's immediate availability, resulting in numerous futile phone calls during urgent breakdowns.

Furthermore, the traditional system lacks standardized pricing, leading to frequent price gouging and arbitrary fee inflation during emergency situations. Safety remains an acute vulnerability: homeowners possess no verifiable means to validate the background, identity, or professional competency of unknown service personnel entering their residences. In the event of substandard workmanship, overcharging, or property damage, customers have no formal grievance mechanism or customer support channel for restitution.

2.2 Proposed system

The proposed system, **NearFix**, addresses these systemic limitations through a cohesive, responsive web platform uniting customers, service providers, and administrators into an integrated ecosystem. Built on a performant single-page application framework, NearFix provides interactive geospatial discovery, enabling customers to discover verified technicians located within their immediate geographical radius using interactive map markers.

NearFix introduces dual booking paradigms: standard scheduled reservations for planned maintenance and instant emergency dispatch for critical incidents. Detailed technician portfolios showcase past project images, verified skills, transparent hourly wages, and authentic customer ratings. For technicians, NearFix functions as a comprehensive digital workbench featuring incoming job queues, interactive calendar schedule management, and instant status updates. An administrative back-office oversees system integrity through KYC audit pipelines, automated billing calculation, and formal dispute mediation, establishing an unprecedented benchmark in hyperlocal service delivery.

2.3 Module Description

The NearFix project is architected into 3 primary operational modules:

Admin:

- Administrative Authentication & Secure Login
- System Metrics & Analytic Dashboard Monitoring
- Service Category & Skill Taxonomy Management
- Worker KYC Document Verification & Audit Pipeline
- Worker Profile Approval, Rejection, and Account Suspension
- Customer & Worker Account Administration
- Booking & Dispatch Lifecycle Monitoring
- Dispute & Complaint Resolution Center
- Audit Logs & System Health Review

User (Customer):

- Customer Account Registration & JWT Authentication
- Profile Setup, Contact Details, & Address Management
- Categorical Service Browsing (Plumbing, Electrical, AC, Carpentry)
- Geospatial Map Discovery with Leaflet Marker Visualization
- Technician Profile Inspection & Work Portfolio Showcase
- Dual Booking Initiation: Scheduled or Emergency Dispatch
- Real-Time Booking Status Lifecycle Tracking
- Service Cost Calculation & Invoicing Breakdown
- Post-Service Rating & Review Submission
- Filing Formal Complaints with Photo Evidence

Service Worker:

- Technician Registration & Identity Authentication
- Profile Configuration: Hourly Rates, Service Modes, & Bio
- KYC Document Upload (Aadhaar, Address Proof, Certificates)
- Portfolio Management (Past Projects, Photos, & Descriptions)
- Schedule Management & Busy Slot Calendar Blocking
- Live Job Requests Queue (Pending, Accepted, Active)
- Job Acceptance or Rejection with Reason Specification
- Real-Time Status Progression (On The Way, Arrived, In Progress, Completed)

2.4 Sprint

SPRINT 1

Module	Task	Hours for completion	Expected date of completion	Actual date of completion
Admin / User / Worker	Role selection & JWT authentication	2 hours	23/07/2024	23/07/2024
	Password hashing & session security	1 hours	23/07/2024	23/07/2024
Admin	Manage service categories & tags	2 hours	06/08/2024	06/08/2024
	Worker KYC verification interface	4 hours	06/08/2024	06/08/2024
	Manage & view registered workers	3 hours	06/08/2024	06/08/2024
Worker	Profile setup & KYC upload	4 hours	13/08/2024	13/08/2024
	Portfolio project showcase	5 hours	13/08/2024	13/08/2024
User	Search & category filter UI	4 hours	13/08/2024	13/08/2024
	Leaflet map marker visualization	6 hours	20/08/2024	20/08/2024
Admin	View system audit & bookings	3 hours	20/08/2024	20/08/2024

SPRINT 2

Module	Task	Hours for completion	Expected date of completion	Actual date of completion
User	Standard booking request form	3 hours	27/08/2024	27/08/2024
User	Emergency dispatch & surge calculation	4 hours	27/08/2024	27/08/2024
Worker	Accept or reject job request	3 hours	27/08/2024	27/08/2024
Worker	Schedule calendar & busy slots	4 hours	03/09/2024	03/09/2024
Worker	Update job progression status	5 hours	03/09/2024	03/09/2024
User	View live booking status timeline	2 hours	10/09/2024	10/09/2024
User	Invoicing & service bill summary	3 hours	10/09/2024	10/09/2024
User	Post-service review & rating	3 hours	17/09/2024	17/09/2024
User	Lodge dispute complaint with evidence	4 hours	17/09/2024	17/09/2024
Admin	Dispute resolution & account suspension	6 hours	24/09/2024	24/09/2024

2.5 User Stories

As an **admin**, the ability to oversee platform integrity and ensure safety across the marketplace is essential. I require a dedicated KYC verification dashboard where I can inspect uploaded government identity credentials, address records, and technical certifications submitted by prospective service workers, approving qualified candidates or rejecting invalid submissions with clear feedback. Additionally, managing service taxonomies (such as Electrician, Plumber, Appliance Care, Painter) allows me to maintain catalog accuracy. Overseeing active bookings and managing customer dispute complaints enables prompt mediation of disputes, issuing warnings, or suspending rogue accounts when regulations are violated.

As a **customer (user)**, the ability to rapidly discover reputable, verified service providers within my immediate geographical radius is vital. I want to seamlessly register, log in, and view available trade professionals on an interactive map or filtered directory list. Viewing authentic worker ratings, hourly pricing, service categories, and verified skill badges enables informed hiring choices. During emergencies such as burst pipes or electrical faults, I require a one-click emergency dispatch request that prioritizes rapid arrival. Following job completion, accessing transparent invoice breakdowns and submitting star ratings and narrative reviews helps sustain platform quality for fellow community members.

As a **service worker**, possessing an autonomous digital storefront is crucial to expanding my client base and building professional reputation. I need to easily manage my profile, define hourly billing rates, specify my geographical service radius, and upload identity credentials for verification. Receiving real-time job notifications allows me to accept or decline service requests based on availability. To avoid scheduling conflicts, I require an interactive calendar to mark personal busy slots. Throughout an active job, updating my status (from On The Way to Completed) keeps the client informed, while showcasing completed project photos on my portfolio builds enduring client trust.

FEASIBILITY STUDY

3. FEASIBILITY STUDY

A comprehensive feasibility study assesses the operational, economic, technical, behavioral, and software viability of the proposed **NearFix** platform prior to extensive engineering commitments. Evaluating these multi-dimensional factors ensures the system fulfills commercial objectives while maintaining operational resilience, cost prudence, and technological scalability.

3.1 Economic feasibility

The NearFix marketplace exhibits exceptional economic viability. Traditional service matchmaking models impose excessive financial overheads involving commercial call centers, physical franchised offices, and print marketing campaigns. In contrast, NearFix leverages a cloud-native, open-source software stack that radically diminishes capital expenditure.

Furthermore, by utilizing open-source Leaflet and OpenStreetMap rendering rather than proprietary map APIs that levy recurring query charges, recurring operational expenses are kept near zero. For technicians, joining the marketplace costs nothing upfront, replacing expensive local advertising with a performance-driven reputation index. For platform administrators, sustainable monetization can be unlocked through modest convenience charges on emergency dispatches, creating a highly cost-effective and lucrative business model.

3.2 Technical Feasibility

The technical feasibility of NearFix is solidly anchored in modern, industrial-grade web engineering standards. The frontend utilizes **React 19** and **Vite**, delivering instantaneous compilation, reactive component rendering, and zero-latency single-page navigation. The backend RESTful API is implemented with **Node.js** and **Express.js**, which excels at high-throughput asynchronous I/O and concurrent request handling.

Data persistence is managed through **MongoDB** with **Mongoose ODM**, providing dynamic schema validation for complex polymorphic entities such as bookings, user roles, KYC documents, and project showcases. In addition, an embedded in-memory database fallback ensures faultless, zero-configuration local deployment, proving complete technical feasibility.

3.3 Operational Feasibility

Operationally, NearFix is streamlined for maximum friction reduction. Both customers

3.4 Behavioral Feasibility

User acceptance is fundamental to marketplace adoption. NearFix is architected with a human-centric philosophy tailored to dual user demographics: tech-savvy homeowners seeking instantaneous utility, and independent trade workers who require a straightforward, uncomplicated interface.

To overcome technical apprehension among local artisans, the worker interface employs clean visual cards, recognizable iconography (via Lucide React), and concise status toggles (e.g., "Available / Busy", "Accept / Reject"). For homeowners, transparent pricing models, verified worker badges, and authentic customer testimonials dispel prevailing safety fears and skepticism regarding home service providers. Rigorous feedback mechanisms and admin-moderated complaint resolutions guarantee behavioral confidence, mutual trust, and long-term user retention.

3.5 Software Feasibility

The NearFix software architecture adheres to robust modular software engineering principles, ensuring compatibility, security, and effortless extensibility across heterogeneous client environments. The web application is engineered under responsive viewport standards, delivering flawless usability across modern desktop web browsers (Chrome, Edge, Firefox, Safari) and mobile touch displays.

Stateless session security is enforced through cryptographic JSON Web Tokens (JWT) coupled with salted bcrypt password hashing, shielding sensitive user data against interception or credential scraping. Decoupled architectural boundaries between the React presentation tier and the Express service tier ensure that feature enhancements, payment gateway integrations, or native mobile wrappers can be incorporated without destabilizing the core operating logic. By maintaining standards-compliant code, comprehensive error guards, and clean component encapsulation, the software demonstrates complete technical durability and long-term maintainability.

SOFTWARE ENGINEERING PARADIGM

4. SOFTWARE ENGINEERING PARADIGM

The software engineering paradigm, also commonly designated as the Software Development Life Cycle (SDLC) model, represents the structured development strategy encompassing technical processes, methodologies, and architectural tools. It governs the evolutionary trajectory of a software artifact from conceptual formulation to requirements elicitation, design modeling, implementation, rigorous verification, and operational maintenance.

4.1 Agile model

The **Agile Model** is an incremental and iterative software development methodology that prioritizes flexibility, team collaboration, autonomous iteration, and rapid customer feedback integration. Rather than adhering to rigid, monolithic specifications characteristic of sequential waterfall models, Agile decomposes software delivery into compact, manageable development iterations known as sprints.

Throughout the construction of NearFix, Agile facilitated rapid responses to evolving marketplace requirements. Iterative cycles allowed continuous refinement of the booking lifecycle, real-time UI adaptations for technician schedule blocking, and incremental enhancement of the administrative KYC verification pipeline. Continuous integration and testing ensured that functional increments remained testable, reliable, and deployable throughout the engineering lifespan.

4.2 Scrum

Scrum is an established, highly structured Agile framework specifically engineered to manage complex, dynamic software initiatives. Scrum organizes engineering tasks into fixed-duration intervals called sprints, typically spanning two to three weeks. Development is guided through clearly delineated roles: the Product Owner (defining feature backlogs and user stories), the Scrum Master (facilitating impediments removal and process adherence), and the Cross-Functional Development Team.

NearFix adopted Scrum ceremonies including Sprint Planning (allocating discrete user stories), Daily Standups (tracking daily milestones and blockers), Sprint Reviews (demonstrating verifiable increments such as Leaflet marker rendering or booking dispatch), and Sprint Retrospectives (optimizing development throughput). This rigorous cadenced methodology guaranteed timely feature delivery, superior architectural stability, and total visibility across all development deliverables.

SYSTEM REQUIREMENT SPECIFICATION

5. SYSTEM REQUIREMENTS SPECIFICATION

5.1 Software requirements

Operating system	Windows 10 / 11, Linux (Ubuntu 20+), macOS
Frontend Framework	React 19, Vite, React Router DOM v7
Frontend Styling & Icons	Vanilla CSS (Glassmorphism), Lucide React
Geospatial Mapping	Leaflet, React-Leaflet, OpenStreetMap
Backend Server & Runtime	Node.js (v18+), Express.js (v5)
Database & Modeling	MongoDB (v6+), Mongoose (v9)
Embedded Fallback DB	mongodb-memory-server (v11)
Authentication & Security	JSON Web Token (JWT), bcryptjs
File Upload Middleware	Multer (v2)
Development IDE & Tools	VS Code, Postman, Chrome DevTools

5.2 Hardware requirements

Processor	Intel Core i3 / AMD Ryzen 3 or above (2.0 GHz+)
RAM	4 GB minimum (8 GB recommended)
Storage Space	10 GB free disk space & above
Display Resolution	1366 x 768 or higher
Network Interface	Standard Ethernet / Wi-Fi broadband connection

SYSTEM DESIGN

6. SYSTEM DESIGN

System design is the foundational phase in the engineering life cycle of a complex software application. It represents the disciplined process of applying architectural principles and technical models to define system components, modules, interfaces, data relationships, and operational flows with sufficient structural fidelity to permit its physical realization and deployment.

6.1 Database Design

Database design is the systematic process of producing a robust, normalized, and performant data model capable of supporting the business operations of the application. In NearFix, the data persistence tier models heterogeneous interactions involving multi-role users, geographical coordinates, complex multi-state service requests, multimedia portfolio items, and dispute records.

Normalization

Normalization is the formal technique of structuring relational schemas and document models to eliminate data anomalies, reduce redundant storage footprints, and safeguard referential integrity across concurrent operations.

Normal Forms:

These are formal criteria for structuring relations to eliminate insertion, deletion, and update anomalies:

1. First Normal Form (1NF):

A relation is in first normal form if every domain attribute contains only atomic, indivisible values, and there are no repeating groups or nested arrays of unstructured attributes.

2. Second Normal Form (2NF):

A relation satisfies 2NF if it is in 1NF and all non-key attributes are fully functionally dependent on the primary key, eliminating partial functional dependencies on composite candidate keys.

3. Third Normal Form (3NF):

A relation satisfies third normal form if it conforms to 2NF and exhibits no transitive dependencies—meaning no non-prime attribute depends transitively upon the primary key through another non-prime attribute.

6.2 Tables

USER (CUSTOMERS, WORKERS, ADMINS)

Field name	Type	Width	Constraints
User_Id	ObjectId	24	Primary key
Role	Varchar	15	Not null ('customer','worker','admin')
Name	Varchar	50	Not null
Email	Varchar	50	Unique, Not null
Phone	Varchar	15	Not null
Address	Varchar	150	Not null
Coordinates	Array[Float]	2	Not null [Longitude, Latitude]

LOGIN / AUTH CREDENTIALS

Field name	Type	Width	Constraints
Auth_Id	ObjectId	24	Primary key
User_Id	ObjectId	24	Foreign key (Ref: USER)
Password_Hash	Varchar	60	Bcrypt salted hash, Not null
Last_Login	DateTime	20	Default: Current timestamp

SERVICE CATEGORY

Field name	Type	Width	Constraints
Category_Id	ObjectId	24	Primary key
Category_Name	Varchar	50	Unique, Not null
Icon_Name	Varchar	30	Not null

JOB REQUEST (BOOKING)

Field name	Type	Width	Constraints
Booking_Id	ObjectId	24	Primary key
Customer_Id	ObjectId	24	Foreign key (Ref: USER)
Worker_Id	ObjectId	24	Foreign key (Ref: USER)
Service_Type	Varchar	50	Not null
Service_Date	Varchar	20	Not null
Status	Varchar	25	Not null ('Pending','Accepted','Completed')
Is_Emergency	Boolean	5	Default: false
Total_Amount	Float	10	Not null

NOTIFICATION

Field name	Type	Width	Constraints
Notification_Id	ObjectId	24	Primary key
User_Id	ObjectId	24	Foreign key (Ref: USER)
Type	Varchar	35	Not null
Message	Varchar	200	Not null
Is_Read	Boolean	5	Default: false

REVIEW & RATING

Field name	Type	Width	Constraints
Review_Id	ObjectId	24	Primary key
Booking_Id	ObjectId	24	Unique Foreign key (Ref: BOOKING)
Rating	Int	5	Not null (Range: 1 to 5)
Comment	Varchar	500	Not null

COMPLAINT (DISPUTE)

Field name	Type	Width	Constraints
Complaint_Id	ObjectId	24	Primary key
Customer_Id	ObjectId	24	Foreign key (Ref: USER)
Worker_Id	ObjectId	24	Foreign key (Ref: USER)
Category	Varchar	40	Not null ('Overcharging','Misconduct')
Description	Varchar	1000	Not null
Status	Varchar	20	Default: 'Open' ('Under Review','Resolved')

WORKER KYC DOCUMENTS

Field name	Type	Width	Constraints
Doc_Id	ObjectId	24	Primary key
Worker_Id	ObjectId	24	Foreign key (Ref: USER)
Identity_Proof	Varchar	150	Government ID URI, Not null
Address_Proof	Varchar	150	Utility bill / Aadhaar URI
Skill_Certificate	Varchar	150	Trade certificate URI
Verification_Status	Varchar	20	'Pending','Verified','Rejected'

BUSY SLOTS (SCHEDULE)

Field name	Type	Width	Constraints
Slot_Id	ObjectId	24	Primary key
Worker_Id	ObjectId	24	Foreign key (Ref: USER)
Slot_Date	Date	15	Not null
Time_Window	Varchar	30	Not null (e.g. '10:00 AM - 12:00 PM')

PROJECT (WORKER PORTFOLIO)

Field name	Type	Width	Constraints
Project_Id	ObjectId	24	Primary key
Worker_Id	ObjectId	24	Foreign key (Ref: USER)
Title	Varchar	100	Not null
Description	Varchar	800	Not null
Category	Varchar	50	Not null
Media_Url	Varchar	150	Uploaded image / showcase URI
Project_Type	Varchar	30	'Completed Work','Repair','Installation'

AUDIT LOG

Field name	Type	Width	Constraints
Log_Id	ObjectId	24	Primary key
Actor_Id	ObjectId	24	Foreign key (Ref: USER)
Action_Type	Varchar	40	'KYC_APPROVE', 'SUSPEND_USER', 'RESOLVE_DISPUTE'
Target_Entity	Varchar	30	Target collection name
Timestamp	DateTime	20	Default: Current timestamp

6.3 UML Designs

The **Unified Modeling Language (UML)** is an industry-standard graphical visual language designed for specifying, constructing, visualizing, and documenting the architectural artifacts of software systems. UML establishes a rigorous syntactical framework that encapsulates proven software engineering best practices, allowing development teams, system architects, and stakeholders to communicate structural and behavioral design choices with precision.

UML incorporates structural diagrams (such as class and component models) and behavioral diagrams (including use case, sequence, and activity charts). By visually expressing system characteristics, UML reduces conceptual ambiguities, streamlines technical onboarding, and provides an authoritative roadmap during physical software implementation.

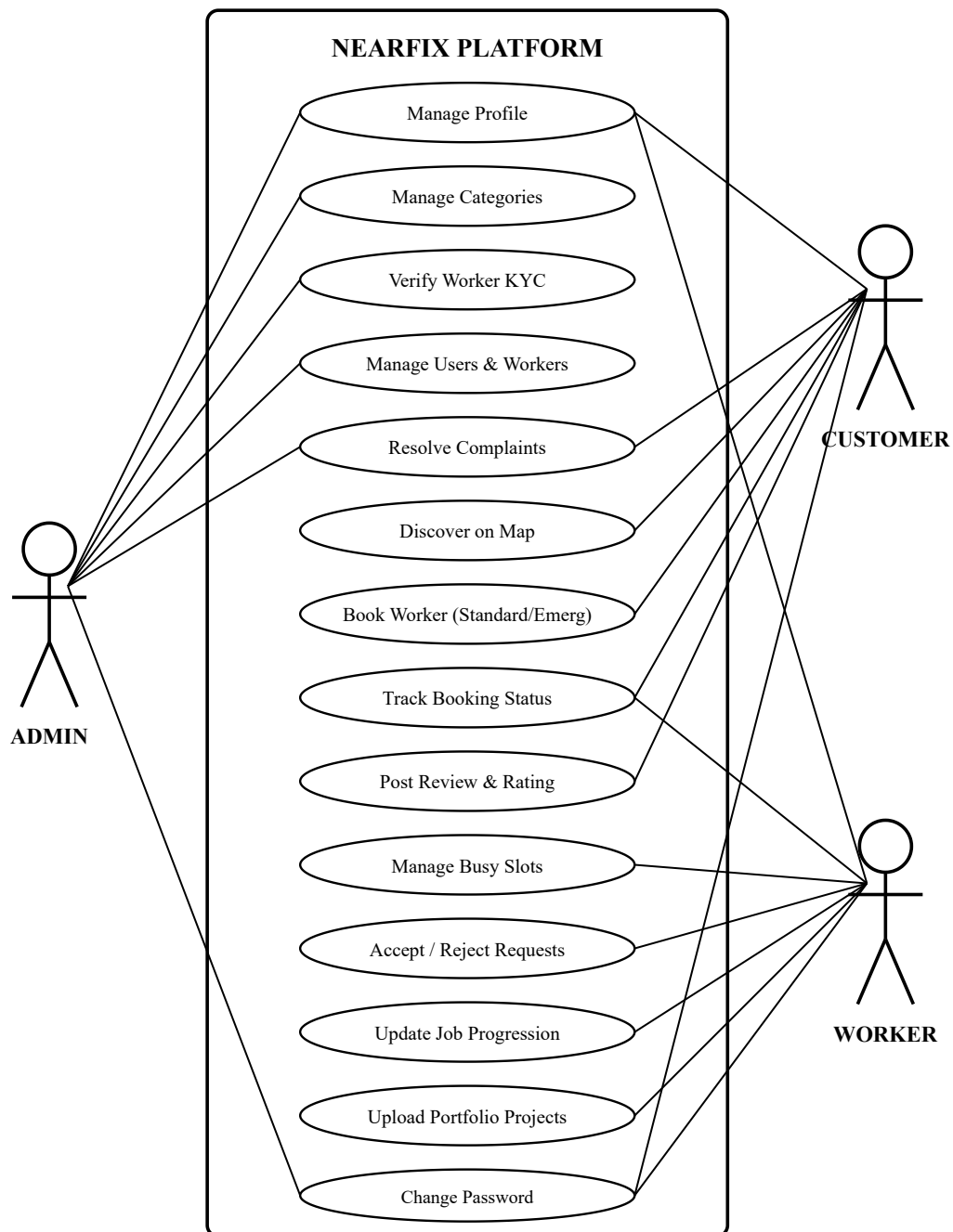
Use Case Diagram

A **Use Case Diagram** provides an abstracted, functional representation of the system modeled strictly from the perspectives of external actors. It depicts the behavioral boundaries of the application and delineates the specific interactions that external entities (human users or automated sub-systems) engage in to accomplish discrete operational objectives. In NearFix, the use case model articulates the respective boundaries, privileges, and common interactions governed across Platform Administrators, Registered Customers, and Skilled Service Workers.

Sequence Diagram

A **Sequence Diagram** is a dynamic interaction diagram that illustrates the chronological message exchanges and invocations passing between participating objects, boundary controllers, and storage entities over an explicit timeline. Sequence diagrams model the execution flow of specific operational scenarios, detailing invocation directions, synchronous and asynchronous message dispatches, return parameters, and entity lifespans.

6.4 Use case diagram



6.5 Scenario

Admin:

- Admin can securely authenticate and log into the administrative management console
- Admin can inspect, verify, or reject worker KYC identity and trade documents
- Admin can add, modify, or remove service classifications and skill tags
- Admin can monitor system metrics, active users, and verified workers
- Admin can review active job dispatches and historical booking records
- Admin can investigate customer dispute complaints and examine photographic evidence
- Admin can record resolution directives and suspend non-compliant worker accounts
- Admin can update credentials and change account passwords

Customer (User):

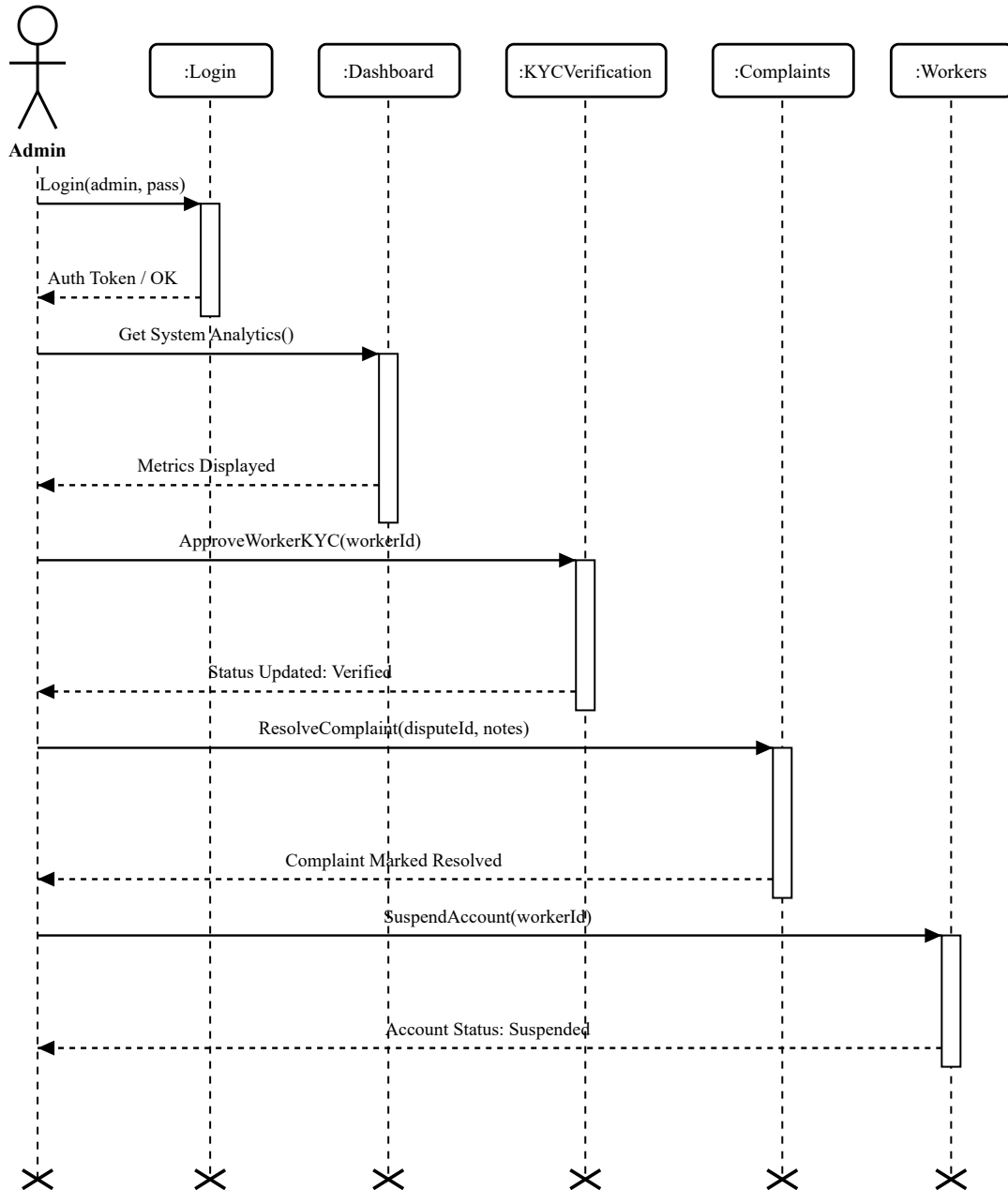
- Customer can register a personal account and securely log in
- Customer can manage home addresses, phone number, and profile details
- Customer can browse verified trade categories and view ratings
- Customer can locate nearby technicians on an interactive Leaflet map
- Customer can book service providers via standard schedule or emergency priority
- Customer can inspect live booking status from dispatch through completion
- Customer can submit star ratings and detailed written reviews
- Customer can file dispute complaints with photographic evidence if dissatisfied
- Customer can update profile settings and change account password

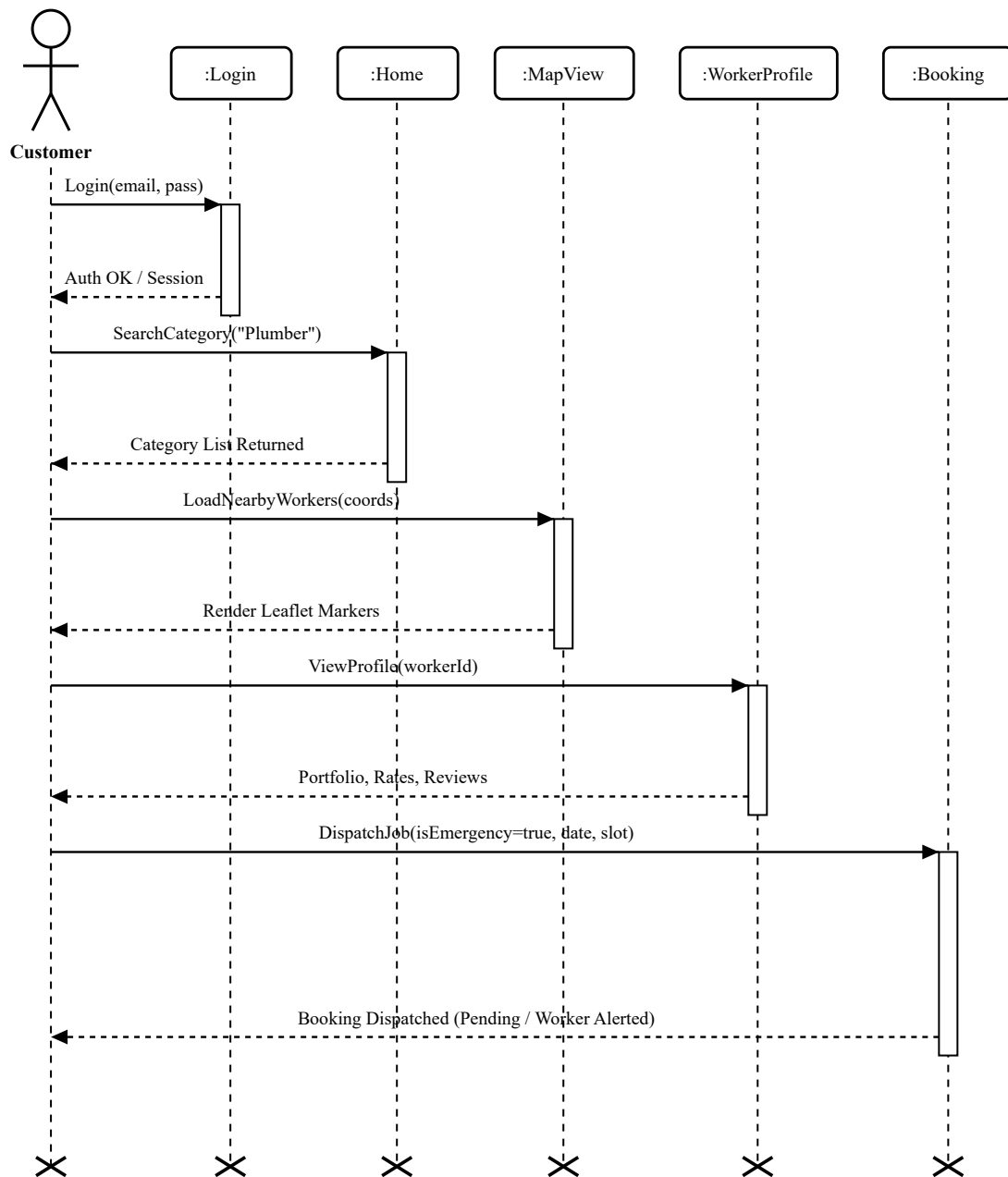
Service Worker:

- Worker can register account and submit KYC documents for platform verification
- Worker can define professional hourly rates, service modes, and trade bio
- Worker can upload past work portfolio photographs and job summaries
- Worker can configure an interactive calendar and mark busy unavailable slots
- Worker can receive incoming job requests and accept or decline based on availability
- Worker can update real-time execution states (On The Way, Arrived, In Progress, Completed)
- Worker can inspect received customer ratings, feedback, and performance metrics
- Worker can manage account settings and update security credentials

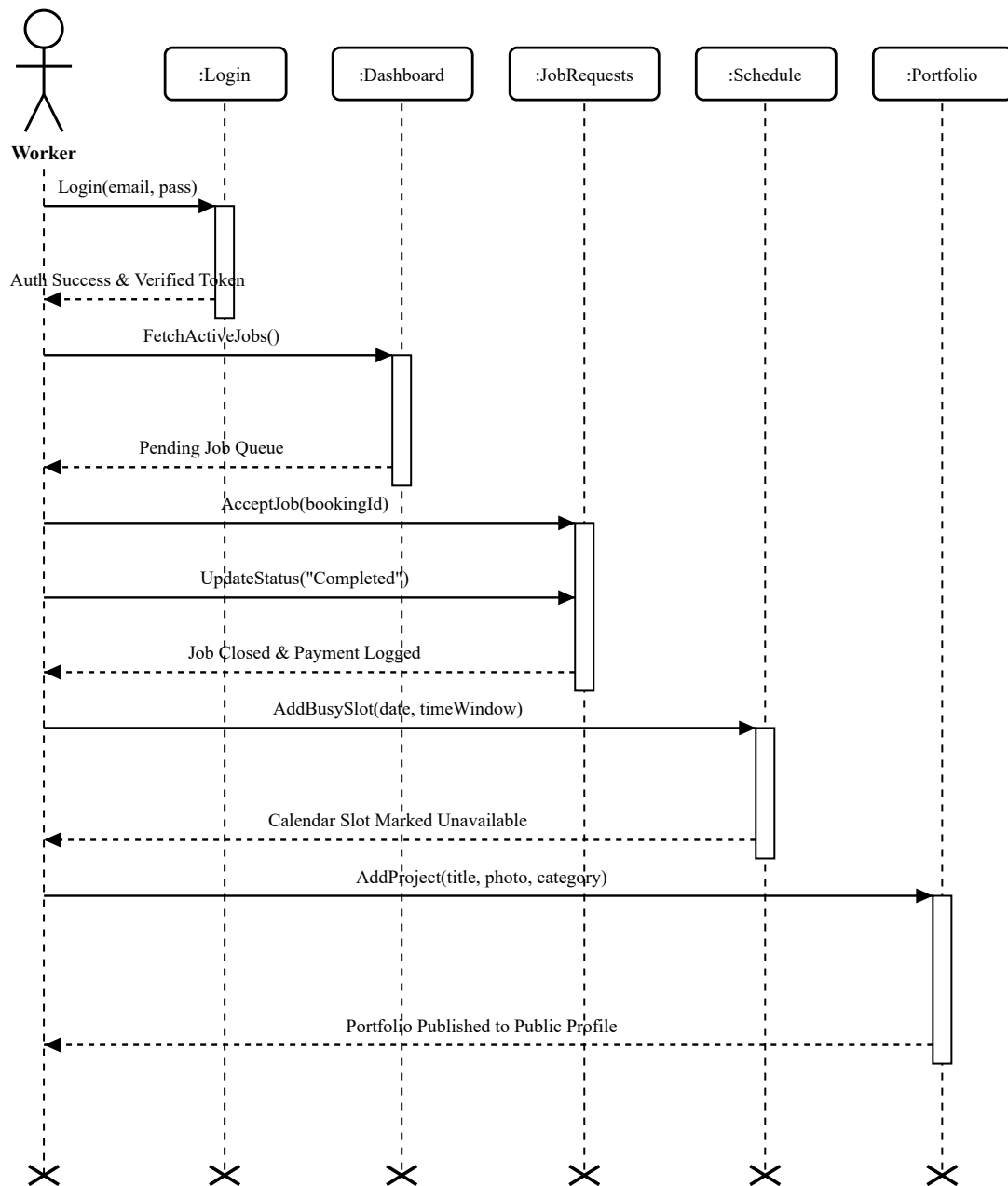
6.6 Sequence Diagram

Admin



Customer (User)

Service Worker



SYSTEM DEVELOPMENT

7. SYSTEM DEVELOPMENT

System development constitutes the operational transformation of conceptual architecture, logical database models, and abstract workflow designs into an executable, stable, and verifiable software product. The development process is partitioned into two major parallel sequences:

- **External System Development:** User interface engineering, client-side rendering, geospatial visual mapping, and responsive presentation layouts.
- **Internal System Development:** Application programming interfaces (APIs), business logic execution, database persistence, session security, and state synchronization.

The major milestones across the development phase comprise:

- Technical Requirements Decomposition & Environment Setup
- Component Architecture & UI State Management
- RESTful Endpoint Synthesis & Middleware Integration
- Automated Database Seed Configuration & Deployment

7.1 Coding

The primary purpose of coding is to express software logic through unambiguous, maintainable, and syntactically sound computer instructions that manipulate data structures to produce required functional outputs.

JavaScript & Node.js Runtime

JavaScript serves as the unified foundational language across both client and server tiers in NearFix. **Node.js** provides an asynchronous, non-blocking, event-driven runtime environment executing the V8 JavaScript engine outside the browser. Node.js enables high-concurrency handling of simultaneous network requests, making it exceptionally suited for real-time dispatching, status transitions, and data streaming.

React 19

React 19 powers the client-side single-page presentation tier. React utilizes a declarative virtual DOM abstraction, enabling reactive state mutations and seamless UI updates without demanding complete browser document reloads. Modular component hierarchies ensure high code reusability, maintainability, and rapid responsiveness across dynamic views.

Express.js Framework

Express.js is a minimalist, flexible web application framework for Node.js. It equips NearFix with robust routing mechanisms, parameter validation, and middleware pipelines. In NearFix, Express coordinates authentication guards (`authMiddleware`), administrative role authorizers (`adminMiddleware`), JSON payload parsing, and error-handling controllers, standardizing RESTful API communication.

MongoDB & Mongoose ODM

MongoDB is a high-performance, document-oriented NoSQL database that persists data in flexible, JSON-like BSON documents. It accommodates evolving data structures effortlessly, facilitating geospatial coordinate queries (`2dsphere` indexes) and embedded document arrays. **Mongoose** acts as the Object Data Modeling (ODM) layer, enforcing strict schema definitions, data casting, default assignments, validation rules, and relational population across collections.

Leaflet Open-Source Mapping

Leaflet is an open-source, mobile-friendly JavaScript library for interactive web maps. Integrated through **React-Leaflet**, it renders OpenStreetMap tile layers with high rendering performance. Leaflet enables dynamic marker placement for verified technicians, custom popup windows, and coordinate jitter algorithms that prevent visual marker collisions at identical neighborhood positions.

HTML5 & Vanilla CSS Design System

HTML5 provides semantic document markup, ensuring structural integrity and accessibility standards. **Vanilla CSS** implements a modern glassmorphism design system utilizing tailored CSS variables, dark-mode gradients, responsive flexbox/grid containers, and subtle micro-animations. This decoupled approach avoids heavyweight external styling frameworks while guaranteeing rapid stylesheet parsing and fluid visual rendering.

SYSTEM TESTING AND IMPLEMENTATION

8. SYSTEM TESTING AND IMPLEMENTATION

Software testing is an essential verification process undertaken to confirm that the implemented system conforms to functional specifications, operates robustly under boundary conditions, and achieves zero-defect stability before deployment into operational environments.

8.1 Types of Testing

Unit testing

Unit testing isolates individual software units—such as standalone controller functions, utility algorithms, or validation rules—and verifies their output correctness against prescribed inputs. In NearFix, unit tests validated password hashing functions, JWT generation routines, coordinate validation, and surge fee computation logic independently of peripheral modules.

Black box testing

Black box testing evaluates the software purely from an external functional perspective without inspecting internal source code or algorithmic paths. Test cases verified user registration, booking dispatches, search filters, and status transitions against expected outputs.

White box testing

White box testing examines the internal logical paths, conditionals, loop boundaries, and exception branches of the software. In NearFix, white box testing validated middleware execution, authorization token validation paths, database connection failover routines, and error-handling responses.

8.2 Implementation

Implementation marks the transition from design constructs to a working software system. The deployment strategy involved containerized server execution, environment configuration via secure variable files (`.env`), database indexing, and build bundle optimization via Vite. Pre-seeded demo datasets enabled immediate stakeholder demonstration and verification.

SYSTEM MAINTENANCE

9. SYSTEM MAINTENANCE

Software maintenance constitutes the post-delivery modification of a software system to remediate latent defects, adapt to dynamic operating environments, and integrate new functional enhancements throughout its operational lifecycle.

There are 3 fundamental categories of software maintenance:

- **Corrective Maintenance:** Identifying, diagnosing, and repairing undiscovered software bugs, runtime exceptions, or edge-case calculation errors reported by active marketplace users.
- **Adaptive Maintenance:** Modifying the platform to maintain compatibility with external environmental evolutions, including browser engine updates, updated Node.js runtime releases, or revised third-party mapping API standards.
- **Perfective Maintenance:** Implementing functional enhancements, optimizing database query execution, refining UI responsiveness, and integrating new user capabilities that improve overall user satisfaction and system maintainability.

Software maintenance remains the cornerstone of long-term software viability. Real-world systems inevitably encounter unexpected user behaviors and edge cases that escaped formal testing cycles. Furthermore, rapid innovations across the computing landscape necessitate continual updates to security libraries, cryptographic standards, and runtime dependencies.

NearFix embraces modular component design, centralized configuration management, and robust logging frameworks. These architectural practices dramatically reduce maintenance overhead, ensuring the platform can evolve effortlessly while preserving total stability, data integrity, and peak operational reliability over extended operational lifespans.

FUTURE ENHANCEMENT

10. FUTURE ENHANCEMENT

While NearFix provides a highly capable, scalable foundation for hyperlocal on-demand service delivery, several progressive technological enhancements are planned to expand platform reach, operational efficiency, and customer satisfaction:

- **Native Mobile Applications (iOS & Android):** Developing cross-platform mobile apps utilizing React Native to leverage device GPS hardware, native push notifications, and background location telemetry for accurate worker tracking.
- **Automated Escrow Payment Gateways:** Integrating industrial payment solutions (such as Razorpay, Stripe, or UPI Auto-Pay) with escrow smart-holds, releasing payments to technicians strictly upon customer satisfaction sign-off.
- **AI-Powered Smart Dispatch & Matchmaking:** Deploying machine learning algorithms that evaluate technician proximity, historical completion rates, skill proficiency, and current workload to automatically suggest the optimal technician for emergency jobs.
- **Real-Time Video Diagnostic Consultations:** Incorporating WebRTC-based in-app video calling, allowing technicians to remotely inspect plumbing or electrical issues and provide preliminary estimates before physical travel.
- **Multilingual & Vernacular Voice Support:** Offering native language interfaces and voice-command navigation tailored to local artisans and diverse regional customer demographics.
- **IoT Diagnostic Integration:** Connecting smart home appliances directly to the platform to automatically trigger service dispatches upon hardware error detection.

These future enhancements will further elevate NearFix into an autonomous, intelligent, and ubiquitous home maintenance utility that sets new industry standards in hyperlocal service delivery.

CONCLUSION

11. CONCLUSION

In conclusion, the **NearFix** hyperlocal service marketplace delivers an innovative, transformative solution addressing the deep-rooted limitations of the conventional unorganized home maintenance sector. By harnessing modern web engineering practices and prioritizing trust, speed, and transparency, the project delivers a reliable digital bridge connecting local homeowners with skilled micro-entrepreneurs.

The platform successfully fulfills its core objectives. Customers gain access to an intuitive web interface providing geospatial worker discovery, verified technician credentials, authentic peer reviews, and an indispensable dual-booking mechanism supporting standard appointments as well as rapid emergency dispatches. Local trade professionals gain an empowered, autonomous digital storefront complete with visual portfolio showcases, flexible scheduling controls, and streamlined job tracking.

Safety and accountability are firmly embedded throughout the platform via administrative KYC vetting, transparent billing calculations, and formalized dispute resolution channels. By leveraging open-source mapping and scalable MERN stack technologies, NearFix establishes a cost-effective, high-performance architecture capable of seamless geographic expansion.

Ultimately, NearFix redefines the home maintenance experience from an uncertain, stressful chore into a frictionless, transparent, and trustworthy utility. The system serves as a solid technological foundation for future innovations, bridging community needs and driving sustainable economic empowerment for skilled independent artisans.

APPENDIX

12. APPENDIX

Admin Login

NearFix • Admin Portal Authorization

HTTPS://NEARFIX.LOCAL/ADMIN/LOGIN

⚡

NearFix Security Gateway

Enter credentials to access administrative dashboard

Email Address:

admin@nearfix.com

Password:

••••••••

Sign In to Console

Admin Dashboard Analytics

Admin Overview • Metrics & Real-Time Telemetry

SYSTEM ANALYTICS

Total Users

1,420

Verified Workers

385

Active Bookings

42

KYC Pending

7

Worker KYC Verification Queue

Administrative KYC Vetting & Verification Pipeline				KYC WORKFLOW
Worker Name	Trade / Skill	Documents Uploaded	Status	Actions
Rajesh Kumar	Master Electrician	Aadhaar ID, Wireman License	Pending	ApproveReject
Mohammed Shaan	Sanitary Plumber	Voter Card, Plumbing Cert	Verified	View

Admin Dispute & Complaint Center

Customer Grievance & Arbitration Dashboard				RESOLUTION PIPELINE	
Complaint ID	Customer	Worker	Category	Status	Action
#CMP-8812	Anjali Nair	Suresh V	Overcharging	Under Review	<button>Resolve</button>
#CMP-8805	Rahul Das	Manoj K	Poor service	Resolved	<button>Closed</button>

Service Category & Taxonomy Management

Manage Service Classifications & Tag Catalog

TAXONOMY CONFIG

+ Add Category

Electrician (48 workers)

Plumbing (36 workers)

AC & HVAC (29 workers)

Carpentry (22 workers)

Painting (18 workers)

Deep Cleaning (31 workers)

Admin Worker Account Moderation & Suspension

Worker Profile Oversight & Status Enforcement

MODERATION

Suresh V (Appliance Repair) • Rating: 2.1 ★ (3 Complaints)

Aadhaar ID: 7748-9921-0012 • Status: Flagged for Misconduct

Suspend Account

Customer Registration Form

NearFix • Create Customer Profile		REGISTRATION
Full Name:	Email Address:	
Anjali Nair	anjali@example.com	
Phone Number:	Service Location (Pincode):	
+91 98471 23456	Calicut (673008)	
Create Account		

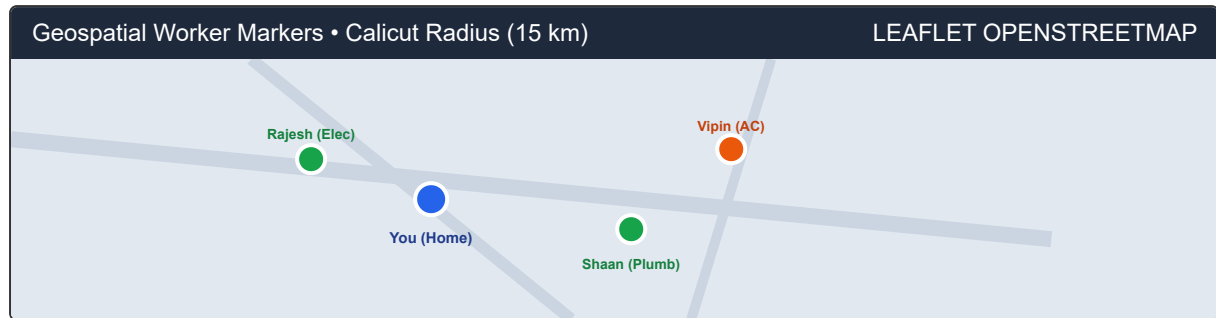
Customer Home & Category Browsing

NearFix • Instant Home Services at Your Doorstep		CUSTOMER PORTAL
Reliable Help, Verified Locally		
Discover vetted technicians within 15 km of your location		
Search electricians, plumbers, painters in Calicut...		
 Electricians	 Plumbers	 AC Repair
		 Painters

Customer Profile & Saved Addresses

Customer Account Details & Preferences	PROFILE
Anjali Nair (anjali@example.com) Primary Address: House #14, Silver Crest Apts, Kuttikkattoor, Calicut	Edit Details

Interactive Leaflet Map Discovery



Worker Profile & Portfolio Preview

Technician Storefront • Rajesh Kumar (Master Electrician) VERIFIED PROFILE

Rajesh Kumar ✓ KYC VERIFIED
8+ Years Exp • Starting: ₹350/hr • 4.8 ★ (82 reviews)
Specialties: Domestic Rewiring, Inverter Setup, Fault Diagnostics

Book Emergency

Booking Dispatch Modal (Dual Options)

Schedule Appointment or Request Emergency Dispatch DISPATCH MODAL

Standard Booking
Select preferred date and morning/afternoon slot
Base: ₹350

Emergency Dispatch
Guaranteed arrival within 45 mins + surge fee
Total: ₹550 (₹350 + ₹200 surge)

Customer Active Bookings & History

Track Real-Time Status & Historical Services				MY BOOKINGS
Booking ID	Service Type	Worker	Mode	Status
#NF-1049	Electrical Short Circuit	Rajesh Kumar	Emergency	On The Way
#NF-1022	Pipe Leakage Repair	Shaan M	Standard	Completed

Invoice Summary & Transparent Fee Breakdown

Generated Invoicing & Bill Receipt		TAX INVOICE
Base Service Fee (Hourly Consultation):		₹350.00
Emergency Priority Surge Charge:		+ ₹200.00
Total Payable Amount:		₹550.00

Customer Review & Rating Submission

Post-Service Experience Feedback

REVIEW SUBMISSION

Rate your experience with **Rajesh Kumar**:

★★★★★ (5.0 / 5.0)

"Arrived within 25 minutes during our electrical emergency! Solved the issue very professionally."

Submit Review

Worker Portal Dashboard

Technician Control Center • Rajesh Kumar

WORKER DASHBOARD

Completed Jobs
84

Monthly Earnings
₹31,400

Client Rating
4.8 ★

Incoming Job Requests Queue

Live Booking Alerts • High Priority Emergency Dispatch

JOB QUEUE

EMERGENCY ALERT

Electrical Sparking in Circuit Breaker

Customer: Anjali Nair • Location: Kuttikkattoor (2.2 km away)

Est. Payout: ₹550 (Includes ₹200 Emergency Surge)

Accept Job

Decline

Real-Time Job Progression Controls

Execution State Machine • Active Job #NF-1049

LIFECYCLE CONTROLS

Accepted ✓

On The Way

Arrived

In Progress

Mark Completed

Worker Calendar Schedule & Busy Slot Blocking

Manage Daily Working Hours & Personal Availability		SCHEDULE MANAGER	
Today: Wednesday, 28 Aug		+ Mark Slot Busy	
09:00 - 11:00 Available	11:00 - 01:00 Busy (Booked)	02:00 - 04:00 Available	04:00 - 06:00 Busy (Personal)

Worker Portfolio Project Showcase Upload

Proof of Work • Upload Past Completed Projects		PORTFOLIO MANAGER
3-Phase Distribution Board Setup Completed in Palazhi, Calicut • Category: Electrical ✓ Verified Showcase Item	+ Upload New Project Photo / Video	

Worker Reviews & Performance Scorecard

Customer Ratings & Written Feedback		REPUTATION SCORECARD
Rahul Das "Extremely fast arrival and high quality wiring. Very polite and reasonably priced."		★★★★★

BIBLIOGRAPHY

13. BIBLIOGRAPHY

Websites

- [1] *React Official Documentation*, <https://react.dev/learn>
- [2] *Node.js v18+ API Reference Documentation*, <https://nodejs.org/docs/latest/api/>
- [3] *Express.js Web Application Framework Documentation*, <https://expressjs.com/>
- [4] *MongoDB Manual & Mongoose ODM Schema Architecture*, <https://mongoosejs.com/docs/>
- [5] *Leaflet - Modern Open-Source JavaScript Maps Library*, <https://leafletjs.com/reference.html>
- [6] *MDN Web Docs - HTML5, Cascading Style Sheets, and JavaScript Standards*, <https://developer.mozilla.org/>

Books

- [1] *Full Stack React with TypeScript*, Nate Murray, Felipe Coury, Ari Lerner, CreateSpace, 2021
- [2] *Node.js Design Patterns (3rd Edition)*, Mario Casciaro and Luciano Mammino, Packt Publishing, 2020
- [3] *MongoDB: The Definitive Guide (3rd Edition)*, Shannon Bradshaw, Eoin Brazil, Kristina Chodorow, O'Reilly Media, 2019
- [4] *Clean Architecture: A Craftsman's Guide to Software Structure and Design*, Robert C. Martin, Prentice Hall, 2017
- [5] *Agile Software Development: Principles, Patterns, and Practices*, Robert C. Martin, Pearson Education, 2013
- [6] *Web Application Security: Exploitation and Countermeasures for Modern Web Applications*, Andrew Hoffman, O'Reilly Media, 2020